Informe Técnico de Arquitectura

Implementación Física de Base de Datos & Análisis de Motor SGBD

SGBD: PostgreSQL 16+
GUI: pgAdmin 4

1. Justificación Técnica de PostgreSQL 16 y pgAdmin 4

Para la construcción del sistema relacional de la Red Social Estudiantil e integración del módulo ambiental IoT, se ha seleccionado el Motor de Base de Datos Relacional Orientado a Objetos **PostgreSQL 16+** junto con su cliente gráfico oficial **pgAdmin 4**. Esta combinación fue elegida debido a su probada estabilidad industrial, soporte completo del estándar ANSI SQL y capacidad para gestionar cargas híbridas relacionales y NoSQL mediante el tipo de dato binario optimizado JSONB.

Transacciones ACID

Garantiza aislamiento estricto (MVCC) y consistencia en escrituras concurrentes de interacciones y publicaciones.

Soporte Híbrido NoSQL

Almacenamiento e indexación de objetos JSONB mediante índices invertidos GIN para telemetría Big Data e IoT.

Gestión pgAdmin 4

Entorno gráfico completo para auditoría de planes de ejecución (EXPLAIN), administración de esquemas y Query Tool.

2. Guía Paso a Paso de Implementación en pgAdmin 4

A continuación se detalla la secuencia técnica efectuada dentro del entorno gráfico pgAdmin 4 para desplegar la estructura física desde el script DDL:

1. **Conexión al servidor PostgreSQL:** Inicio de sesión en el cliente pgAdmin 4 mediante la autenticación del superusuario `postgres` en el puerto local `5432`.
2. **Creación de la base de datos objetivo:** Generación de la base de datos nombrada `red_social_estudiantil` configurada con codificación de caracteres `UTF-8` para garantizar soporte de caracteres especiales.
3. **Apertura de la Consola SQL (Query Tool):** Selección del nodo de la base de datos e ingreso a la interfaz de edición de código ejecutable.
4. **Ejecución del Script DDL:** Carga y compilación de las instrucciones de creación de tablas, llaves primarias, foráneas con borrado en cascada y restricciones CHECK.
5. **Inspección y Validación del Catálogo:** Verificación de la presencia de las tablas en el esquema `public` mediante consultas al catálogo `information_schema.columns`.

3. Análisis Comparativo: Ventajas, Desventajas y Dificultades

Durante la implementación práctica en pgAdmin 4 se evaluaron las prestaciones operativas del motor frente a otros SGBD tradicionales:

Criterio Evaluado	Análisis Técnico y Observaciones de Campo
Ventajas Observadas	Sólido cumplimiento de restricciones referenciales integradas, excelente soporte para consultas JSONB complejas e inclusión de secuencias modernas `GENERATED ALWAYS AS IDENTITY`.
Desventajas Detectadas	Consumo de memoria RAM superior en el cliente pgAdmin 4 al ser una interfaz basada en la web local, y estricta necesidad de definir el orden de creación/borrado de tablas por dependencias de llaves foráneas.
Dificultades Técnicas	Manejo de la relación autorreferenciada/bidireccional de la tabla `conexiones`, resuelta mediante restricciones `CHECK (id_solicitante <> id_receptor)` y clave primaria compuesta.

4. Tipos de Datos SQL y NoSQL para Big Data e IoT

Optimización de Almacenamiento e Ingesta Masiva:
En escenarios IoT donde sensores ambientales generan miles de capturas por minuto, el tipo `REAL` (4 bytes) reduce un 50% el consumo de espacio comparado con `DOUBLE PRECISION` (8 bytes) sin perder precisión física. Asimismo, el tipo `JSONB` permite almacenar cargas semiestructuradas dinámicas (payloads NoSQL) que pueden indexarse mediante índices **GIN** para búsquedas inmediatas en milisegundos.



5. Conclusiones Individuales y Aprendizajes

El desarrollo de este proyecto permitió afianzar la transición práctica desde un diagrama conceptual Entidad-Relación hasta un esquema relacional optimizado físicamente en PostgreSQL 16. La definición explícita de restricciones de integridad referencial nombradas y el uso de borrados en cascada (`ON DELETE CASCADE`) demostró ser un mecanismo indispensable para mantener la coherencia de la base de datos sin depender de lógica externa a nivel de aplicación.

6. Propuestas de Mejora y Escalabilidad Futura

Particionamiento por Fechas

Implementar la clausula `PARTITION BY RANGE (fecha_captura)` en la tabla de telemetría para dividir las lecturas en particiones mensuales, optimizando las consultas analíticas sobre grandes volúmenes de datos.

Vistas Materializadas

Crear una `MATERIALIZED VIEW` que precalcule las estadísticas de reacciones y comentarios por publicación, reduciendo la carga de cálculo en tiempo real sobre el servidor principal.

Firma y Validación Técnica:
Documento preparado por **Jerónimo Betancur Betancur** para la asignatura *Bases de Datos I*. Todos los scripts e instrucciones DDL fueron probados y ejecutados satisfactoriamente en PostgreSQL 16 sobre el entorno pgAdmin 4.